

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-a2ad72099f51e3c7e.jsonl`
- SHA-256: `7a6d1684230ce37d5a6a695f55a6c22852d02a2f97f06590f754066f05795637`
- Source modified: `2026-06-22T09:47:50+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`
```

Transcript

1. user (2026-06-22T09:46:17.570Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
  "title": "MEDIUM: Email failure in fail_email trap could silently succeed if ~/.venv doesn't exist",
  "file": "deploy/gcp/nightly_regression/run_on_vm.sh",
  "line": "39-51",
  "severity": "medium",
  "detail": "The fail_email trap (line 52) is set before .venv is activated. If an early failure occurs (e.g., before line 68 which activates .venv), fail_email attempts to activate .venv with . .venv/bin/activate but this fails silently due to 2>/dev/null on line 42. The python command then runs in the current shell (not the venv), which may not have the google-cloud-secret-manager or smtplib modules installed, causing the Python stdin redirect (<<PY) to fail. This is masked by || true on line 44. The owner receives NO failure alert, thinking the run will self-delete successfully.",
  "fix": "Add explicit error handling: check [ -f .venv/bin/activate ] before sourcing it in fail_email. If .venv doesn't exist, log a FATAL message and use system python with a fallback (no secret manager, basic SMTP only). Or: defer fail_email trap setup until AFTER .venv is active (move trap setup to line 68)."
```

2. assistant (2026-06-22T09:46:19.305Z)

I'll analyze this finding by examining the relevant files under the specified absolute path.

3. user (2026-06-22T09:46:20.275Z)

```
1      #!/usr/bin/env bash
2      # Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then DELETES the VM
3      # (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a startup-script, not
```

```

4      # an interactive session). Reads its config from instance metadata (set by launch.sh).
5      #
6      # Flow: Ops Agent ? pull repo @ HEAD from GCS ? build venv + WASB env ? stage
weights ?
7      #      run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report)
?
8      #      upload report to GCS ? SELF-DELETE (even on failure, via the EXIT trap).
9      #
10     # Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-
prefix,
11     # weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a
real VM first.
12     set -uo pipefail
13     exec > >(tee -a /var/log/nightly-regression.log) 2>&1
14     echo "==== nightly-regression startup $(date -u +%FT%TZ) ====="
15     BOOT_EPOCH=$(date +%s)
16
17     md() { curl -s -H "Metadata-Flavor: Google"
"http://metadata.google.internal/computeMetadata/v1/$1"; }
18     NAME=$(md instance/name)
19     ZONE=$(md instance/zone | awk -F/ '{print $NF}')
20     REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
21     GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
22     BUCKET=$(md instance/attributes/gcs-bucket || true)
23     REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
24     WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
25     DO_EMAIL=$(md instance/attributes/email || echo true)
26     SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
27     SMTP_USER=$(md instance/attributes/smtp-user || true)
28     EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
29     DATE=$(date -u +%F)
30
31     # ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM
behind.
32     cleanup() {
33         echo "==== self-delete $NAME ($ZONE) ====="
34         gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
35             echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete
$NAME --zone=$ZONE -q"
36     }
37     trap cleanup EXIT
38
39     fail_email() { # best-effort failure alert (the report email won't have fired on an
early crash)
40         echo "RUN FAILED ? attempting a failure alert"
41         if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42             cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43                 NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44                 python - <<PY || true
45         from scripts.nightly_email import send_report
46         send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
47             "The nightly engine-regression run failed on the VM before producing a
report.\n"
48             "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
49         PY
50         fi
51     }
52     trap 'fail_email' ERR
53
54     # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
55     curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
56         sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"

```

```

57
58     # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
59     mkdir -p ~/rally && cd ~/rally
60     gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
61     tar -xzf /tmp/rally.tgz -C ~/rally
62     echo "$GIT_SHA" > ~/rally/GIT_SHA
63
64     # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README §4).
65     sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
66     export TMPDIR=/scratch 2>/dev/null || true
67     ./deploy/digitalocean/bootstrap.sh --gpu
68     . .venv/bin/activate
69     pip install -e '[storage-gcs]'
70     bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
71
72     # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
73     if [ -n "$WEIGHTS_URI" ]; then
74         mkdir -p ~/models/WASB-SBDT/pretrained_weights
75         gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
76         export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
77     fi
78
79     # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80     UPTIME=$(( (date +%s) - BOOT_EPOCH ))
81     EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82     set +e
83     NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84     python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
85     RUN_RC=$?
86     set -e
87     echo "runner exit code: $RUN_RC (0 pass / 1 regression / 4 stale)"
88
89     # 6) Upload the dated report to GCS for the record (alongside the email).
90     if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
91         gcloud storage cp -r "output/nightly_reelcount/$DATE."*
"gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
92         gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/"
|| true
93     fi
94
95     echo "==== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
96     trap - ERR # disable the failure-email trap; the EXIT (self-delete) trap stays
97     # the EXIT trap deletes the VM
98

```

4. user (2026-06-22T09:46:20.943Z)

```

1     ""Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3     Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
4     dependency-light (stdlib ``smtplib`` + ``email.message``) and ``graceful``: if no SMTP
password can
5     be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
6     never mask a regression).
7
8     Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:

```

```

9      ? Recipient: ``--email-to`` arg ? ``NIGHTLY_EMAIL_TO`` ? ``avi.dullu@gmail.com``
10     ? SMTP host: ``NIGHTLY_SMTP_HOST`` (default ``smtp.gmail.com``) . port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11     ? SMTP user: ``NIGHTLY_SMTP_USER`` (default = recipient)
12     ? SMTP pass: ``NIGHTLY_SMTP_PASS`` ? Secret Manager ``NIGHTLY_SMTP_SECRET``
13     (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-
secret-manager
14     lib if installed, else the ``gcloud`` CLI). A Gmail app password is
the simplest.
15
16     Set the secret once (owner): echo -n '<app-password>' | gcloud secrets create nightly-
smtp-pass --data-file=-
17     and on the VM: export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18     export NIGHTLY_SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19     ""
20     from __future__ import annotations
21
22     import logging
23     import os
24     from email.message import EmailMessage
25     from typing import Optional
26
27     log = logging.getLogger("nightly_email")
28
29     DEFAULT_TO = "avi.dullu@gmail.com"
30     DEFAULT_HOST = "smtp.gmail.com"
31     DEFAULT_PORT = 587
32
33
34     def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) ->
EmailMessage:
35     """A text/plain email carrying the markdown report (renders fine in any client).
Pure ? testable."""
36     msg = EmailMessage()
37     msg["Subject"] = subject
38     msg["From"] = from_addr
39     msg["To"] = to_addr
40     msg.set_content(body_md)
41     return msg
42
43
44     def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
45     """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
unresolvable."""
46     env = env if env is not None else os.environ
47     direct = env.get("NIGHTLY_SMTP_PASS")
48     if direct:
49         return direct
50     secret_ref = env.get("NIGHTLY_SMTP_SECRET")
51     if not secret_ref:
52         return None
53     # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
54     try:
55         from google.cloud import secretmanager # type: ignore
56         client = secretmanager.SecretManagerServiceClient()
57         return
client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
58     except Exception: # noqa: BLE001 ? lib absent or auth issue ? try the CLI
59         pass
60     try:
61         import subprocess
62         name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in
secret_ref else secret_ref

```

```

63         out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--
secret", name],
64                                 capture_output=True, text=True, timeout=30)
65         return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
None
66     except Exception as e: # noqa: BLE001
67         log.warning("secret-manager resolution failed: %s", e)
68         return None
69
70
71     def send_report(subject: str, body_md: str, to: Optional[str] = None,
72                   env: Optional[dict] = None) -> bool:
73         """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
74         env = env if env is not None else os.environ
75         to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
76         host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
77         port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
78         user = env.get("NIGHTLY_SMTP_USER") or to_addr
79         password = resolve_secret(env)
80         if not password:
81             log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
82             return False
83
84         import smtplib
85         msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
86         with smtplib.SMTP(host, port, timeout=60) as s:
87             s.starttls()
88             s.login(user, password)
89             s.send_message(msg)
90         log.info("emailed report to %s", to_addr)
91         return True
92

```

5. user (2026-06-22T09:46:21.613Z)

```

1     # Tiny launcher image for the nightly engine-regression Cloud Run JOB. It is NOT the GPU
workload ?
2     # it only stages the repo + creates the ephemeral GPU VM (which self-deletes). Scale-to-
zero: the job
3     # runs for a few seconds nightly, then exits. Base = google/cloud-sdk (gcloud + git
preinstalled).
4     FROM gcr.io/google.com/cloudsdktool/cloud-sdk:slim
5     WORKDIR /app
6     # The repo is the build context (run `gcloud builds submit` from the repo root, see
register_scheduler.sh).
7     COPY . /app
8     ENTRYPOINT ["bash", "deploy/gcp/nightly_regression/launch.sh"]
9

```

6. assistant (2026-06-22T09:46:23.916Z)

Now I'll read the remaining files to verify the finding about the .venv activation and error handling:

7. user (2026-06-22T09:46:25.577Z)

```

1     """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3     Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4     it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does

```

```

not change**
5      vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6      trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7      needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8      owner the report, and self-deletes. It is default-additive: a tripwire, not the
promotion gate.
9
10     What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11     ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12     Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13     EXACT (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see $guard).
14     ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15     ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16     ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18     Extensible pattern (add a video = one manifest row + re-freeze):
19     ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20     ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22     Usage
23     -----
24     Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25     python scripts/nightly_reelcount.py --update-baseline
26
27     Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28     python scripts/nightly_reelcount.py --email
29
30     Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31     (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32     """
33     from __future__ import annotations
34
35     import argparse
36     import datetime as _dt
37     import json
38     import os
39     import sys
40     from dataclasses import dataclass, field
41     from typing import Any, Dict, List, Optional, Tuple
42
43     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44     if REPO_ROOT not in sys.path:
45         sys.path.insert(0, REPO_ROOT)
46
47     # billing is pure (typing only) ? safe to import at module top.
48     from backend import billing # noqa: E402
49
50     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")

```

```

52     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54     #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56     DEFAULT_QUALITY_TOL = 0.02      # quality is float/labeled ? a looser tripwire than the
exact reel-count
57     DEFAULT_FX_INR_PER_USD = 83.0
58     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59     DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62     @dataclass(frozen=True)
63     class Tolerances:
64         quality_tol: float = DEFAULT_QUALITY_TOL
65
66         def to_dict(self) -> Dict[str, Any]:
67             return {"quality_tol": self.quality_tol}
68
69
70     # ----- #
71     # Pure cost math (wraps backend.billing ? no IO).
72     # ----- #
73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                    fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds x machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79         rate_per_sec = float(machine_usd_per_hr) / 3600.0
80         usage = {billing.WALL_SECONDS: float(billed_seconds)}
81         rates = {billing.WALL_SECONDS: rate_per_sec}
82         usd, breakdown = billing.compute_cost_usd(usage, rates)
83         return {
84             "billed_seconds": round(float(billed_seconds), 1),
85             "gpu_seconds": round(float(gpu_seconds), 1),
86             "machine_usd_per_hr": float(machine_usd_per_hr),
87             "usd": usd,
88             "inr": billing.to_inr(usd, fx_inr_per_usd),
89             "breakdown": breakdown,
90         }
91
92
93     # ----- #
94     # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
95     # ----- #
96     def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                           cost: Dict[str, Any]) -> Dict[str, Any]:
98         """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}}None,
wall_seconds``.

```

8. user (2026-06-22T09:46:26.212Z)

```

1     #!/usr/bin/env bash
2     # Launch ONE nightly engine-regression run on an ephemeral GCP GPU VM, then return. The
VM runs
3     # run_on_vm.sh as its startup-script (regression ? email ? SELF-DELETE), so this just
stages the repo
4     # and kicks the VM off. Driver-agnostic: call it from Cloud Scheduler?Cloud Run job, a

```

```

cron on any
5      # always-on box, the owner's machine, or a Claude routine. Needs gcloud auth + the repo
checkout.
6      #
7      #   bash deploy/gcp/nightly_regression/launch.sh                                # L4, stages HEAD,
kicks the VM
8      #   RALLY_GPU=t4 RALLY_REF=$(git rev-parse HEAD) bash
deploy/gcp/nightly_regression/launch.sh
9      #
10     # ? Run this MANUALLY once as a dry-run (watch the serial log) before wiring a scheduler
? the VM-side
11     # flow is NOT covered by CI. Tail: gcloud compute instances get-serial-port-output
<name> --zone=<z>.
12     set -uo pipefail
13
14     PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
15     BUCKET="${RALLY_BUCKET:-khelesutra-rally-corpus}"
16     SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
17     GPU="${RALLY_GPU:-l4}"
18     NAME="${RALLY_VM_NAME:-rally-nightly-$(date -u +%Y%m%d)}"
19     REF="${RALLY_REF:-HEAD}"
20     WEIGHTS_URI="${RALLY_WEIGHTS_URI:-gs://$BUCKET/weights/wasb_badminton_best.pth.tar}"
21     REPORT_PREFIX="${RALLY_REPORT_PREFIX:-nightly/reports}"
22     DO_EMAIL="${RALLY_EMAIL:-true}"
23     SMTP_SECRET="${NIGHTLY_SMTP_SECRET:-projects/$PROJECT/secrets/nightly-smtp-
pass/versions/latest}"
24     SMTP_USER="${NIGHTLY_SMTP_USER:-}"
25     EMAIL_TO="${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
26     HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
27     STARTUP="$HERE/run_on_vm.sh"
28     export CLOUDSDK_CORE_DISABLE_PROMPTS=1
29
30     ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a
asia-southeast1-b asia-southeast1-c}"
31     if [ "$GPU" = "l4" ]; then MACHINE="g2-standard-4"; ACCEL=(); else
MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1); fi
32
33     # 1) Stage the repo @ REF to GCS (no GitHub auth on the VM; the worker SA can read the
bucket).
34     SHA=$(git rev-parse "$REF")
35     TGZ_URI="gs://$BUCKET/nightly/repo/${SHA}.tgz"
36     echo "== staging repo $SHA -> $TGZ_URI =="
37     TMP_TGZ=$(mktemp --suffix=.tgz)
38     git archive --format=tar.gz -o "$TMP_TGZ" "$REF"
39     gcloud storage cp "$TMP_TGZ" "$TGZ_URI" --project="$PROJECT"
40     rm -f "$TMP_TGZ"
41
42     # 2) Zone-sweep create the VM (mirrors create_gpu_vm.sh) with run_on_vm.sh + the config
metadata.
43     ERR=$(mktemp)
44     for Z in $ZONES; do
45         echo "== creating $NAME ($GPU) in $Z =="
46         if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
47             --machine-type="$MACHINE" "${ACCEL[@]}" \
48             --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
49             --image-family=common-cu129-ubuntu-2204-nvidia-580 --image-project=deeplearning-
platform-release \
50             --boot-disk-size=120GB --boot-disk-type=pd-balanced \
51             --service-account="$SA" --scopes=cloud-platform \
52             --metadata-from-file=startup-script="$STARTUP" \
53             --metadata=repo-tgz-uri="$TGZ_URI",git-sha="$SHA",gcs-bucket="$BUCKET",report-
prefix="$REPORT_PREFIX",weights-uri="$WEIGHTS_URI",email="$DO_EMAIL",smtp-
secret="$SMTP_SECRET",smtp-user="$SMTP_USER",email-to="$EMAIL_TO" \
54             2>"$ERR"; then
55         echo "? launched $NAME in $Z ? it will run the regression, email $EMAIL_TO, and

```

```

self-delete."
56         echo "    serial log: gcloud compute instances get-serial-port-output $NAME --zone=$Z
--project=$PROJECT"
57         echo "    if it doesn't self-delete: gcloud compute instances delete $NAME --zone=$Z
-q"
58         rm -f "$ERR"; exit 0
59     fi
60     if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then echo "    (no $GPU capacity in
$Z ? next zone)"; continue; fi
61     echo "    create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
62     done
63     echo "? no $GPU capacity in any swept zone ($ZONES). Retry later or RALLY_GPU=t4."
64     rm -f "$ERR"; exit 1
65

```

9. assistant (2026-06-22T09:46:28.491Z)

Now let me verify the exact behavior of the error trap and the .venv activation logic by examining the shell syntax and flow:

10. user (2026-06-22T09:46:30.204Z)

```

fail_email() { # best-effort failure alert (the report email won't have fired on an early
crash)
    echo "RUN FAILED ? attempting a failure alert"
    if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
        cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
        NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
        python - <<PY || true
from scripts.nightly_email import send_report
send_report("[KheISutra nightly] ERROR ? engine regression VM run failed ($DATE)",
    "The nightly engine-regression run failed on the VM before producing a report.\n"
    "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial log.\nGIT_SHA=$GIT_SHA",
to="$EMAIL_TO")
PY
    fi
}
trap 'fail_email' ERR

```

11. assistant (2026-06-22T09:46:32.088Z)

Now let me trace through the logic carefully to understand the actual behavior:

12. user (2026-06-22T09:46:39.486Z)

```

Testing: if cd succeeds but .venv doesn't exist...
.venv MISSING
Running the actual compound command:
Done. Exit code: 0

```

13. assistant (2026-06-22T09:46:43.445Z)

Perfect! Now I've confirmed the actual behavior. Let me trace through what really happens:

When the compound command fails at the ``.venv/bin/activate`` step:

- The `&&` chain stops
- The `python`` command does NOT run
- The `|| true`` catches the error and returns exit code 0

So the email is SILENTLY NOT SENT, but the trap handler exits successfully (due to `|| true``), masking the failure. Let me verify the trap actually gets invoked on early failures:

14. user (2026-06-22T09:46:50.278Z)

About to fail...
TRAP FIRED: fail_email called
This would attempt to email
After the failing command

15. assistant (2026-06-22T09:46:55.066Z)

Good. Now let me check whether the .venv exists on the VM when fail_email might be called. The key question is: at what point in the script could an early failure occur?

Looking at the script flow:

1. Lines 17-29: Metadata reads (unlikely to fail)
2. Lines 32-37: cleanup trap + fail_email definition (no-op)
3. **Line 52: ERR trap installed BEFORE .venv is created**
4. Lines 54-70: Install ops agent, pull repo, **bootstrap.sh at line 67**
5. Line 68: `.venv/bin/activate` first occurs here

Let me check what bootstrap.sh does:

16. user (2026-06-22T09:46:55.920Z)

```
./deploy/digitalocean/bootstrap.sh
```

17. user (2026-06-22T09:46:58.622Z)

```
1      #!/usr/bin/env bash
2      #
3      # Bootstrap a DigitalOcean / Paperspace Ubuntu box to run the rally pipeline.
4      #
5      #   ./deploy/digitalocean/bootstrap.sh           # CPU box (Droplet)           ? M0 /
M1-cpu
6      #   ./deploy/digitalocean/bootstrap.sh --gpu    # GPU box (Paperspace / GPU Droplet) ?
M1 / M3.5
7      #
8      # Mirrors .github/workflows/ci.yml (Ubuntu) so the box matches CI exactly.
9      # STORAGE-FREE + SECRET-FREE: proves the pipeline runs on the box via the mocked
10     # test suite. No bucket, no Gemini key, no WSL required. See
11     ../../docs/DECOUPLED_COMPUTE/.
12     set -euo pipefail
13     GPU=0
14     [ "${1:-}" = "--gpu" ] && GPU=1
15
16     echo ">> [1/4] System libraries (ffmpeg + opencv runtime + the watermark font)"
17     sudo apt-get update
18     # fonts-dejavu-core: the stitcher's watermark uses ffmpeg drawtext with DejaVu ? WITHOUT
it
19     # compile_highlights raises and NO reel is produced (found on the 2026-06-21 cloud dry
run: 22
20     # rallies detected but no reel). Matches deploy/cloudrun/Dockerfile. (Or set
stitching.watermark=false.)
21     sudo apt-get install -y python3 python3-venv python3-pip git ffmpeg libgl1 fonts-dejavu-
core
22     # opencv needs libgl2.0-0; the package name differs across Ubuntu releases (t64 on
24.04).
23     sudo apt-get install -y libgl2.0-0t64 || sudo apt-get install -y libgl2.0-0
24
25     echo ">> [2/4] Python venv (.venv)"
26     python3 -m venv .venv
27     # shellcheck disable=SC1091
28     . .venv/bin/activate
29     python -m pip install --upgrade pip
30
31     echo ">> [3/4] torch (${[ "$GPU" = 1 ] && echo CUDA || echo CPU}) + package (editable,
```

```

dev tooling)"
32     if [ "$GPU" = 1 ]; then
33         # CUDA 12.4 wheels. NOTE: verify your GPU arch is supported by this build.
34         # The Paperspace free M4000 is Maxwell (sm_52) ? old archs are sometimes dropped;
35         # if `torch.cuda.is_available()` is False on an M4000, use a P4000/RTX4000 or pin
36         # an older torch. (docs/DECOUPLED_COMPUTE/03 + 06.)
37         pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
38     else
39         pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
40     fi
41     # Runtime deps + pytest/ruff/mypy. The 'reporting' extra (private obsreport over SSH)
42     # is intentionally omitted ? it is a soft dep and its tests skip when absent.
43     pip install -e .[dev]
44
45     echo ">> [4/4] Device check"
46     python - <<'PY'
47     import torch
48     ok = torch.cuda.is_available()
49     print("torch", torch.__version__, "| cuda_available", ok,
50           "| device", (torch.cuda.get_device_name(0) if ok else "CPU only"))
51     PY
52
53     cat <<'EOF'
54
55     >> Bootstrap complete. Run the storage-free, secret-free smoke (mocked suite):
56
57         . .venv/bin/activate && python -m pytest -q
58
59         Expect ~900 passed. That proves the whole pipeline runs on this box with no GPU,
60         no WSL, no bucket and no API key. (On a GPU box the device check above should
61         print cuda_available True.)
62     EOF
63

```

18. assistant (2026-06-22T09:47:05.524Z)

Excellent! Now I see the issue clearly. Let me trace the precise failure scenario:

1. ****Line 52**** in `run\_on\_vm.sh`: `trap 'fail\_email' ERR` is set BEFORE .venv exists
2. ****Line 67****: `./deploy/digitalocean/bootstrap.sh --gpu` is called, which creates .venv at line 26
3. ****If bootstrap.sh FAILS**** (e.g., apt-get fails, pip install fails, or python3 issues) before completing line 26:
 - The ERR trap fires
 - `fail\_email()` is called
 - At this point, .venv has NOT been created yet
 - `cd ~/rally 2>/dev/null` succeeds (we already did this on line 59)
 - ` .venv/bin/activate 2>/dev/null` ****fails silently**** because .venv doesn't exist
 - The `&&` chain stops
 - `python` never runs
 - `|| true` masks the error
 - ****Owner gets NO email****

Let me verify this is actually possible by checking line 59 vs line 67:

19. user (2026-06-22T09:47:07.136Z)

2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses

```

mkdir -p ~/rally && cd ~/rally
gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
tar -xzf /tmp/rally.tgz -C ~/rally
echo "$GIT_SHA" > ~/rally/GIT_SHA

```

3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per dep

```
sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
export TMPDIR=/scratch 2>/dev/null || true
./deploy/digitalocean/bootstrap.sh --gpu
. .venv/bin/activate
pip install -e '[storage-gcs]'
bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned nonzero ?
continuing"
```

20. assistant (2026-06-22T09:47:14.320Z)

Perfect. Now let me confirm the exact failure scenario by checking the logic more carefully:

```
**SCENARIO: If bootstrap.sh fails at line 67 (e.g., apt-get or pip install fails):**
- We're in `~/rally` (line 59 cd)
- .venv does NOT exist (bootstrap.sh failed before creating it)
- ERR trap fires (because bootstrap.sh exits non-zero, and `set -uo pipefail` is in effect)
- `fail_email()` called
- Check: `[ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]` ? TRUE (both pass)
- Compound command executes:
  - `cd ~/rally 2>/dev/null` ? succeeds (we're already there)
  - `&& . .venv/bin/activate 2>/dev/null` ? **FAILS silently** (file doesn't exist)
- Chain stops here
- `&& python -` ? never runs
- `|| true` ? catches the error, returns 0
```

****Result:**** Email is NOT sent, but trap exits cleanly, and the script continues to line 96.

This is a REAL bug. Let me now check if there are any other places that could cause early failures:

21. user (2026-06-22T09:47:15.121Z)

1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.

```
curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
  sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent install
failed (non-fatal)"
```

2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses

```
mkdir -p ~/rally && cd ~/rally
gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
tar -xzf /tmp/rally.tgz -C ~/rally
echo "$GIT_SHA" > ~/rally/GIT_SHA
```

3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per dep

```
sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
export TMPDIR=/scratch 2>/dev/null || true
./deploy/digitalocean/bootstrap.sh --gpu
```

22. assistant (2026-06-22T09:47:18.368Z)

Yes, there are multiple places that could fail before .venv is created:

- Line 60: `gcloud storage cp` could fail
- Line 61: `tar` could fail
- Line 67: `bootstrap.sh` could fail (most likely)

Now let me double-check the email module to see if it would work without the google-cloud-secret-manager:

23. user (2026-06-22T09:47:19.237Z)

```
def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
```

```

    """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
    unresolvable."""
    env = env if env is not None else os.environ
    direct = env.get("NIGHTLY_SMTP_PASS")
    if direct:
        return direct
    secret_ref = env.get("NIGHTLY_SMTP_SECRET")
    if not secret_ref:
        return None
    # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
    try:
        from google.cloud import secretmanager # type: ignore
        client = secretmanager.SecretManagerServiceClient()
        return client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
    except Exception: # noqa: BLE001 ? lib absent or auth issue ? try the CLI
        pass
    try:
        import subprocess
        name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in secret_ref
    else secret_ref
        out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--secret",
        name],
                               capture_output=True, text=True, timeout=30)
        return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else None
    except Exception as e: # noqa: BLE001
        log.warning("secret-manager resolution failed: %s", e)
        return None

def send_report(subject: str, body_md: str, to: Optional[str] = None,
               env: Optional[dict] = None) -> bool:
    """Send the report. Returns True if sent, False if skipped (no creds) ? never raises on
    missing creds."""
    env = env if env is not None else os.environ
    to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
    host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
    port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
    user = env.get("NIGHTLY_SMTP_USER") or to_addr
    password = resolve_secret(env)
    if not password:
        log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ? email
        skipped.")
        return False

    import smtplib
    msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
    with smtplib.SMTP(host, port, timeout=60) as s:
        s.starttls()
        s.login(user, password)
        s.send_message(msg)
    log.info("emailed report to %s", to_addr)
    return True

```

24. assistant (2026-06-22T09:47:27.631Z)

Perfect. Now I have all the evidence. The email module itself is gracefully designed, but the issue is that it's never imported/called because:

1. When ``.venv/bin/activate`` fails silently (line 42), the ``.python`` command never runs
2. The error is swallowed by ``.|| true``

Let me create a comprehensive final check to show this is definitely a real defect:

****EVIDENCE SUMMARY:****

```

| Point | Evidence |
|-----|-----|
| Trap set before venv | Line 52: `trap 'fail_email' ERR` occurs before line 68 where
`.venv/bin/activate` is first successfully called |
| Early failure scenario | Lines 54-67: Multiple failure points (ops-agent, gcloud cp, tar,
bootstrap.sh) occur BEFORE .venv is created |
| Silent venv activation failure | Line 42: `.` .venv/bin/activate 2>/dev/null` redirects stderr
to /dev/null, so activation failure is silent |
| && chain stops | Line 42-44: If activation fails, the `&&` chain stops immediately; `python`
(line 44) never runs |
| Error swallowed | Line 44: `|| true` catches ANY error from the compound command and returns 0
|
| Result | Email is NOT sent, but trap handler succeeds, masking the entire failure |

```

This is a REAL, MEDIUM-severity defect.

25. user (2026-06-22T09:47:43.659Z)

Structured output provided successfully

26. assistant (2026-06-22T09:47:50.530Z)

Summary

****CONFIRMED REAL DEFECT ? MEDIUM SEVERITY****

The finding is **accurate and real**. The ERR trap installed at line 52 can fire during early bootstrap failures (before .venv is created). When `fail\_email()` attempts to activate a non-existent .venv, the activation fails silently due to `2>/dev/null`, the `&&` chain stops, the `python` command never runs, and `|| true` masks the error. The owner receives **NO failure alert**, thinking the run completed successfully (especially since the vm self-deletes via the EXIT trap regardless).

****Concrete failure paths:****

1. `bootstrap.sh --gpu` fails at system package install (apt-get) ? ERR trap fires ? .venv doesn't exist ? email never sent
2. `gcloud storage cp` fails ? same result
3. `tar -xzf` fails ? same result

The defect is production-critical because a regression detector that silently fails to alert the owner is worse than no detector.

****Recommended fix:**** Either defer the ERR trap until after `.venv` is active (losing early-crash alerts), or add defensive checks in `fail\_email()` to gracefully handle missing .venv and log to the serial output instead of silently failing.